Convolutional and Recurrent Networks

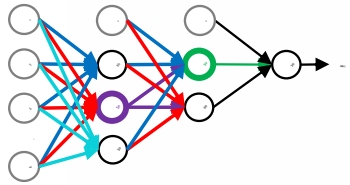
9

Post

CS 3244
Machine Learning

NUS | Computing
National University of Singapore

Recap and Forecast



The Basics of Neural Networks (ANN or NN):

- Explain how neural networks combine perceptrons;
- Explain the topology of a NN;
- Given the index of a weight, find it in a graphical representation of a NN (and vice versa).

- Name potential problems with NNs;
- Describe how a neural network is trained using the backpropagation algorithm;
- Name NN regularization techniques;
- Obtain practical experience constructing NNs using PyTorch

Learning outcomes for this week:

- Understand why deep learning was enabled by certain key technological milestones;
- Understand the Recurrent and Convolutional NN architectures (shortened as RNNs / CNNs);
- Understand how DNNs regularize and optimize via dropout, pretraining and choice of activation function;
- Obtain practical experience in constructing DNNs with PyTorch.

Dissecting the CNN

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

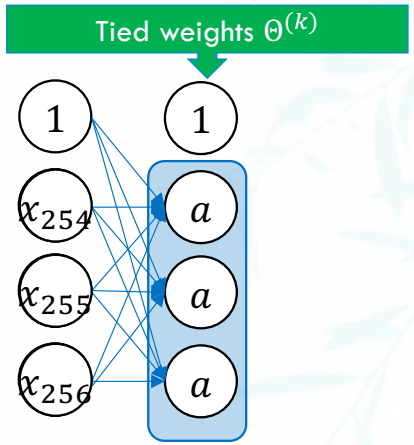
Convolutional Neural Network (CNN)

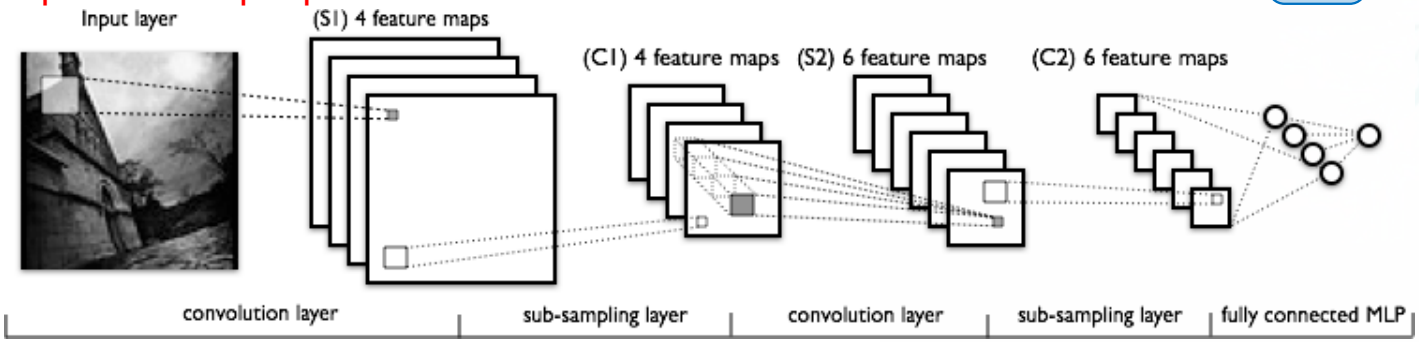
A special case of the general multilayer perceptron (MLP) network, restricted to have fewer parameters.

Applied to images where features are local.

A feature map applies the same learned filter (convolution) over the entire image field

Input layer is not fully connected, but just connected for a particular input patch.

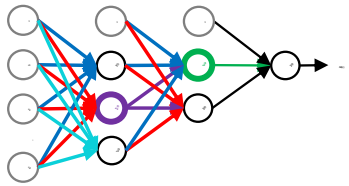


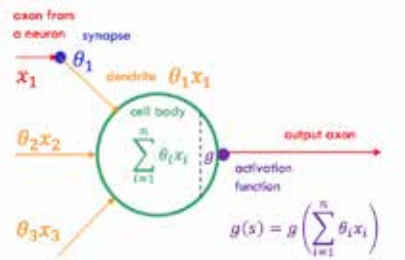


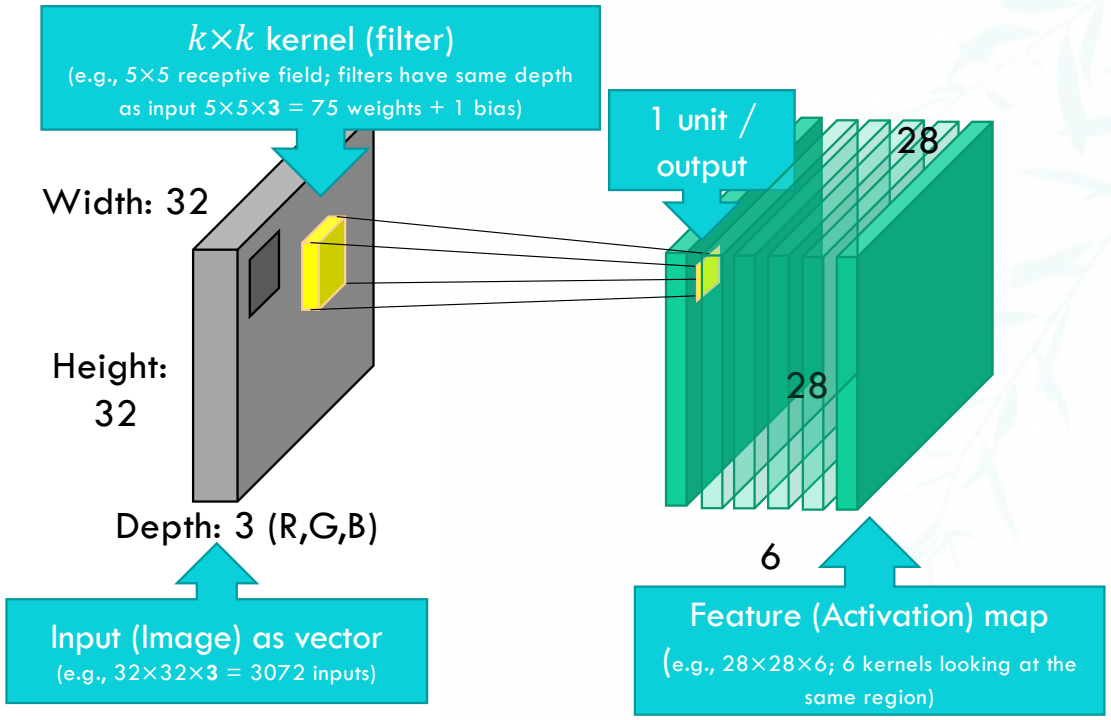
Slide Credits: LeNet Tutorial

Convolution Layer: compiling $s = \sum \theta x$

Traditional Neural Network

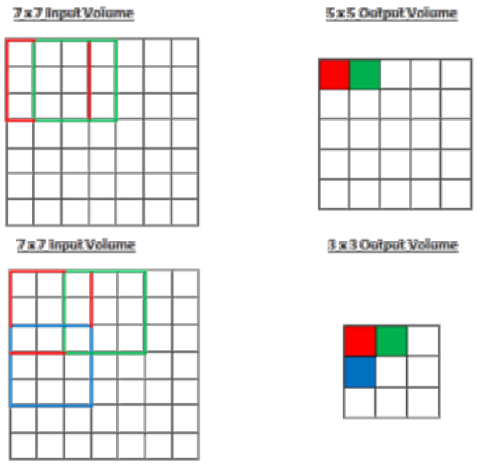




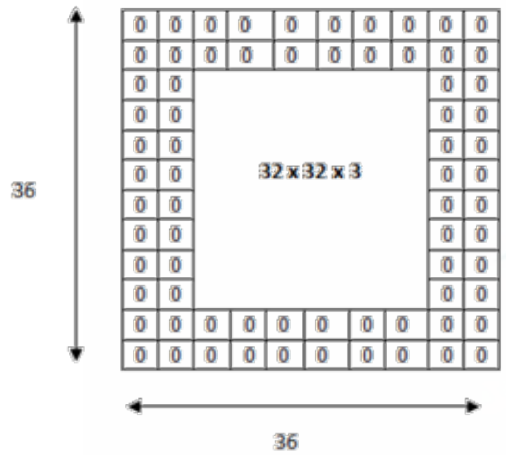


Stride and Padding

Stride: Step size over the image-as-vector



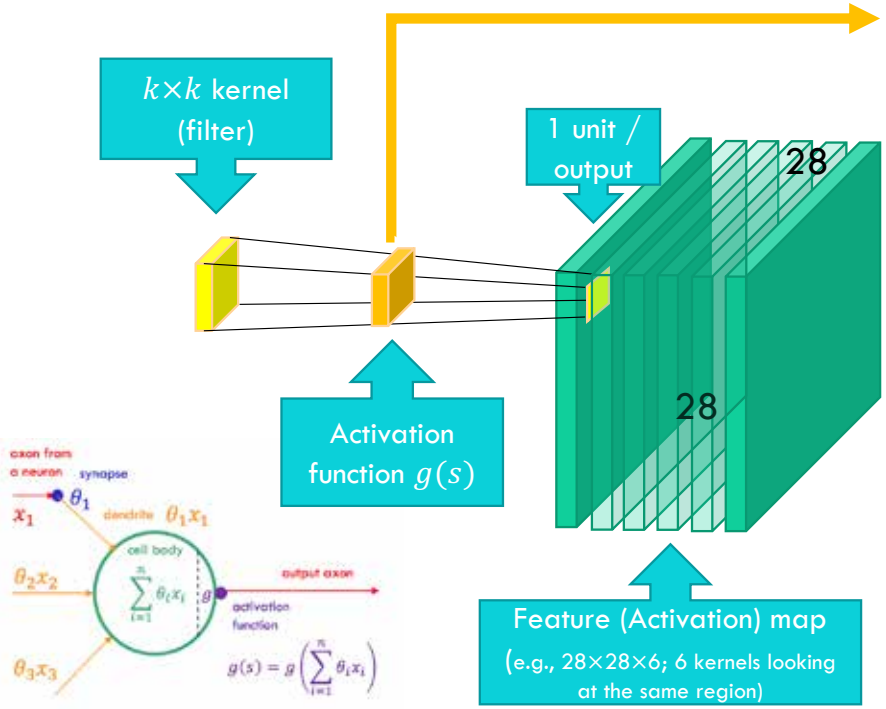
(Zero) padding: to accommodate odd step sizes.



Problem / feature: the image-as-vector shrinks.

Credits: Adit Deshpande. https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks-Part-2/

Non-linear activation layer $g(s)$



Logistic (Sigmoid): $\frac{1}{1+e^{-s}}$

Hyperbolic Tangent: $\tanh(s)$

ReLU: $\max(0, s)$

Leaky ReLU: $\max(0.01s, s)$

PReLU: $\max(\alpha s, s)$

Maxout: $\max_k(\alpha_k s + b_k)$

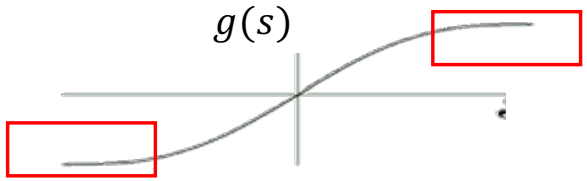
Activation Zoo

Logistic (Sigmoid): $\frac{1}{1+e^{-s}}$

– Not zero centered

Hyperbolic Tangent: $\tanh(s)$

+ Zero centered



– Both saturate at +/– ends. Gradient of cost function saturates (vanishes/explodes) when backpropagating towards input layers.

ReLU: $\max(0, s)$

+ Does not saturate

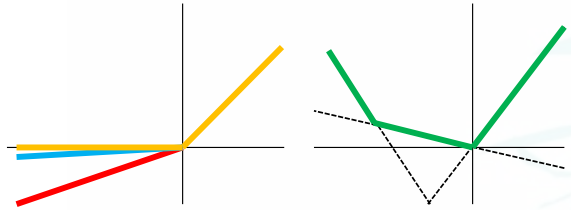
– Units can die, zero output can take a unit off of the network

Leaky ReLU: $\max(0.01s, s)$

+ Leak addresses dying units

PReLU: $\max(\alpha s, s)$

+ parametised, learnable negative portion



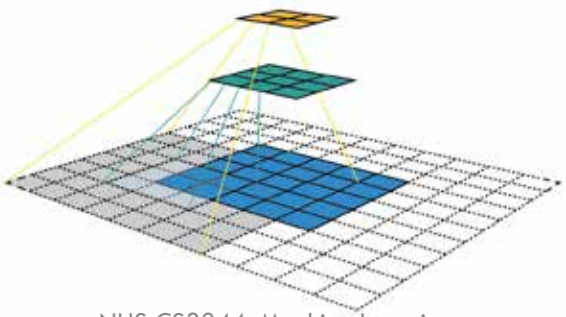
Maxout: $\max_k(\alpha_k s + b_k)$

+ piecewise learnable activation

More complex and general

Pooling Layer

- Downsamples the input; yielding lower resolution input
  - Allows subsequent convolutions to operate over a larger range; increasing receptive field
- Max operation forwards an activation (detection) to the next layer
  - "This feature is somewhere in this region"
  - Usually preferable to averaging, which smooths instead



Output Activation: Softmax

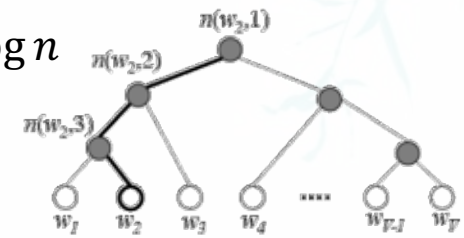
Softmax (Normalized Exponential Function)

- Generalizes logistic activation to multiple classes
- Probabilistic output (in contrast to "hard" max)

$$g(s)_j = \frac{e^{s_j}}{\sum_{k=1}^{K} e^{s_k}}$$

Hierarchical Softmax

- Useful for large # output classes (e.g., natural language)
- Convert flat n way decision into (binary) tree of depth $\log n$
- Parallelizable



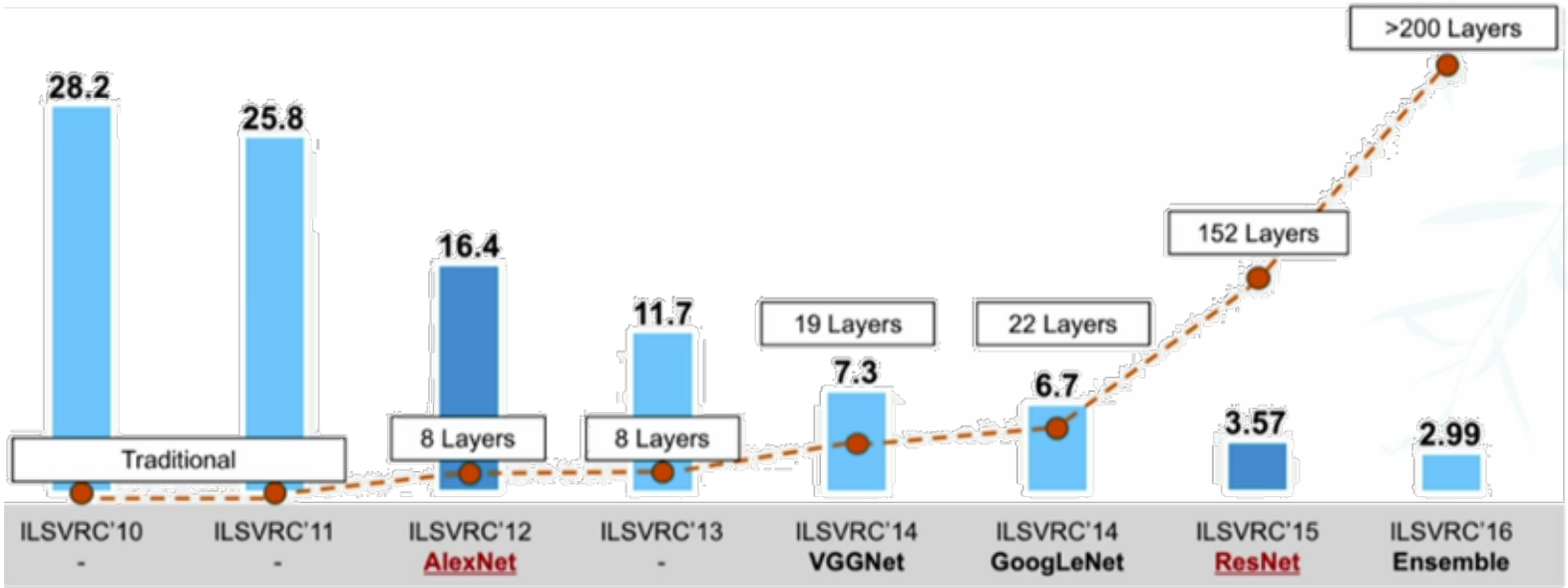
Deepening CNNs

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Kueh Lapis: Deep Layer Cake

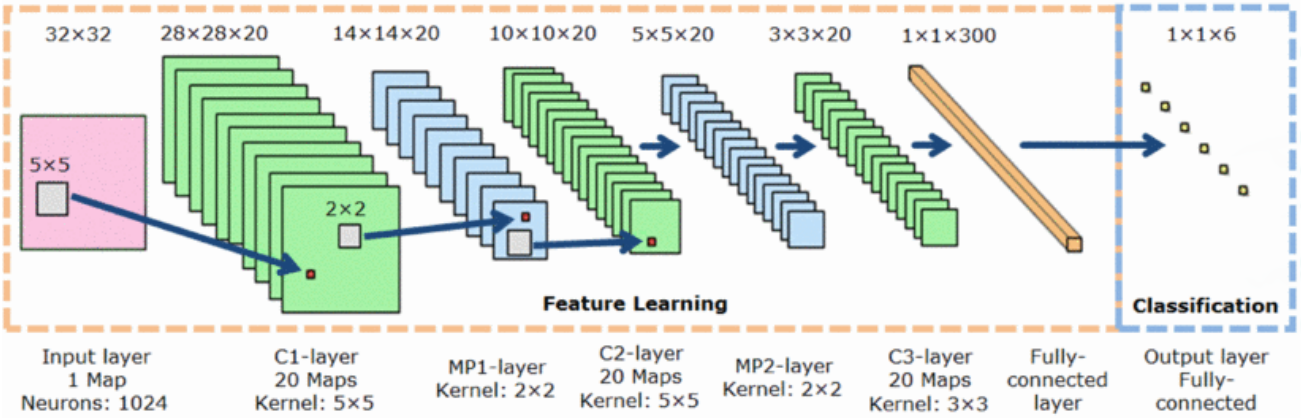
- Bars indicate error rate (lower is better)



Layer Cake

CNNs alternate between (Convolution / Activation) with Pooling layers.

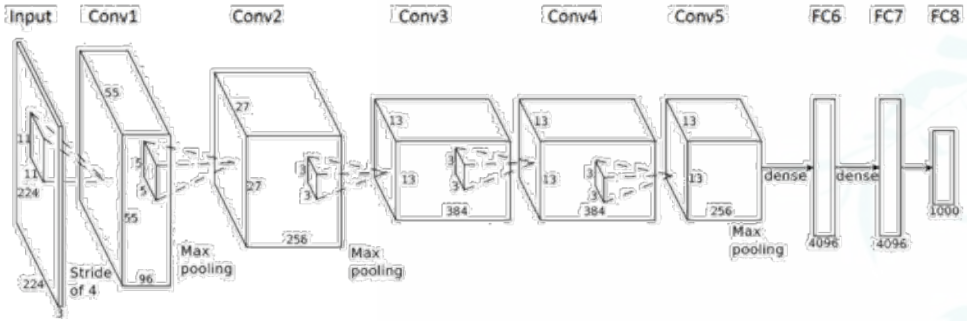
Use final FC layer(s) to allow features to comingle before output.



AlexNet

Winner of 2012 ImageNet Competition (1000 classes)

Full (simplified) AlexNet architecture:
[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

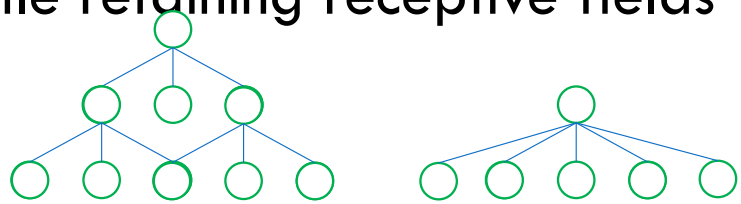


Input: More Memory

Output: More Parameters

VGGNet

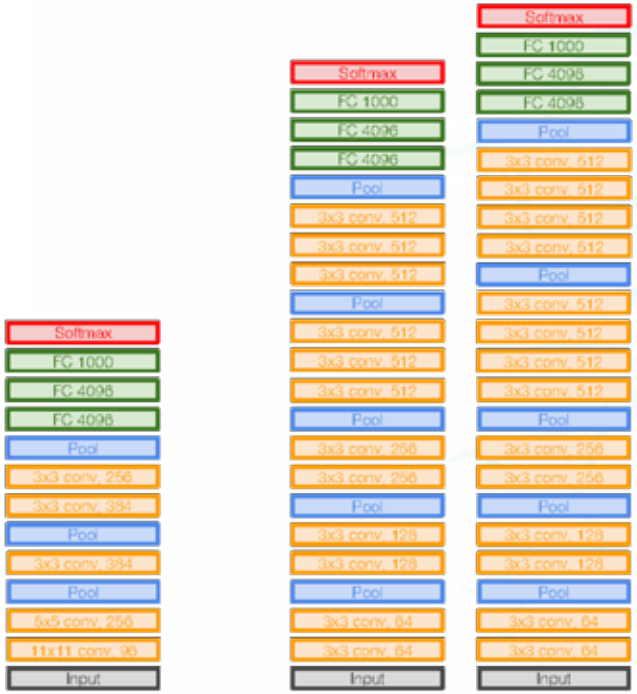
Use smaller convolutions to replace larger ones to gain expressiveness, while retaining receptive fields



stack of 2 3x3 conv
(9+1)×2 parameters

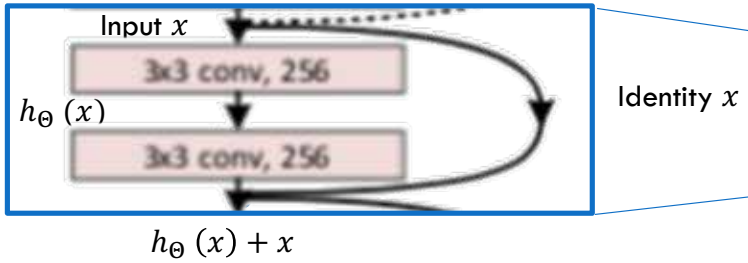
5x5 conv
(25+1) parameters

Depth matters. Better generalization and lower errors.



Residual Network

Build a module that explicit models learning addition nuances uncaptured by previous layers.



Adding more residual modules (depth) won't hurt. Would just learn the identity via the skip connection.

Transfer Learning

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Recursive Feature Extraction

Output layer: task specific

Middle layers: compound features

Lower (closer to input) layers:
simple (edge, intensity) features

Layer 5

Layer 4

Layer 3

Not limited to vision signals

Slide credits: Zeiler and Fergus (2013)

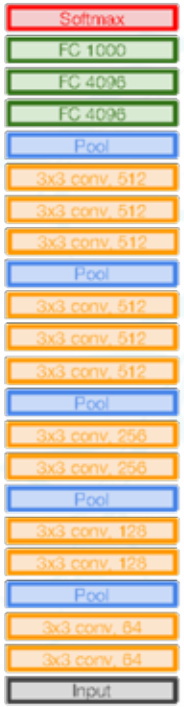
Transfer Learning

Pretrained networks can be used beyond their specific task.

Recipe: Transfer the network for a new task

1. Remove/re-initialize task-specific final layers.

2. Add new final layers, randomly initialized. Train the new layers connected to the body of the previous trained network.



VGG16

Transfer Learning

- (if new task has insufficient data)
  Freeze the body of the network, and only train for the new task on the top layers.

- (if new task has a lot of data)
  Re-train your entire network, but use the previous pre-trained network as an initialization point.

Relearning the same task on a truncated net yields poor performance on final layers, due to fragile co-adaption of features.

VGG16

Time to transfer?

When the two tasks have the same input x.

When the target task B has less data than source task A.

When features extracted for task A is helpful for B.

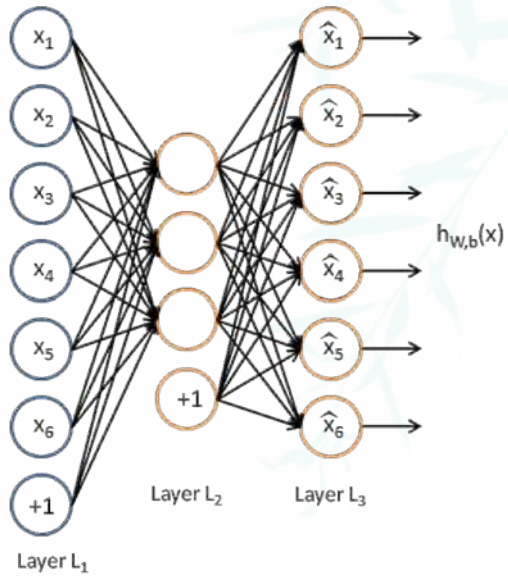
Representation Learning

Use a network in the transfer learning style to create compressed versions of input.

Unsupervised autoencoding 1 hot vectors: Dimensionality reduction.

Adding soft constraints via cost function (Embeddings):

- *Word*: similar words share context
- *Multilingual*: word translations should be near each other
- *Multimodal*: images and their captions should be near each other



RNN Computation

CS3244 Machine Learning

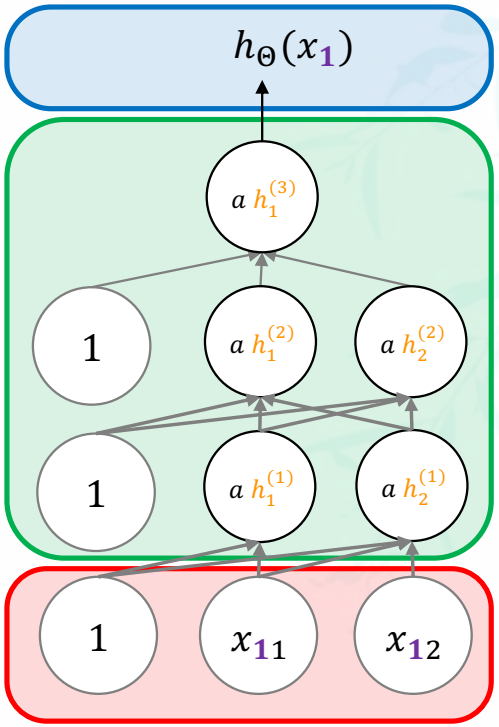
NUS National University of Singapore
Department of Computer Science
School of Computing

Recurrent NNs keep state

In addition to the weights Θ in the network, an RNN incorporates a (hidden) state h, as a vector.

Each unit in the network has a respective dimension of h (unit state).

Randomly initialized, and to be tuned through backprop.



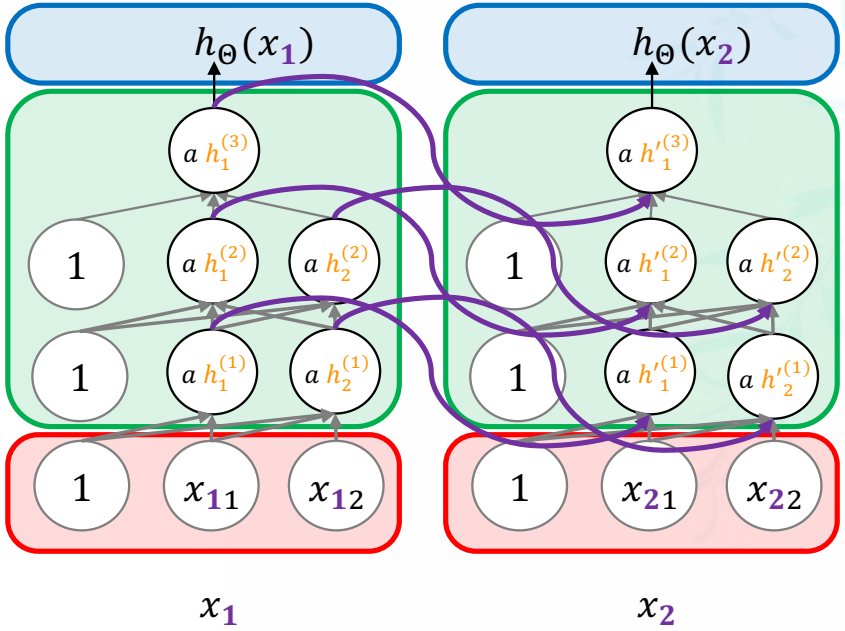
Recurrent NNs keep state

We process a sequence of inputs x by applying the recurrence at each time step.

$$h' = g_\Theta(h, x')$$

Single set of Θ for all time steps.

Weight tying, as in CNNs.



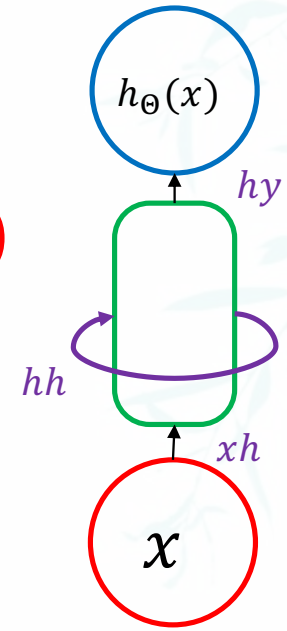
One state, three weights

We then need individual weight matrices to represent each of the three sets of edges.

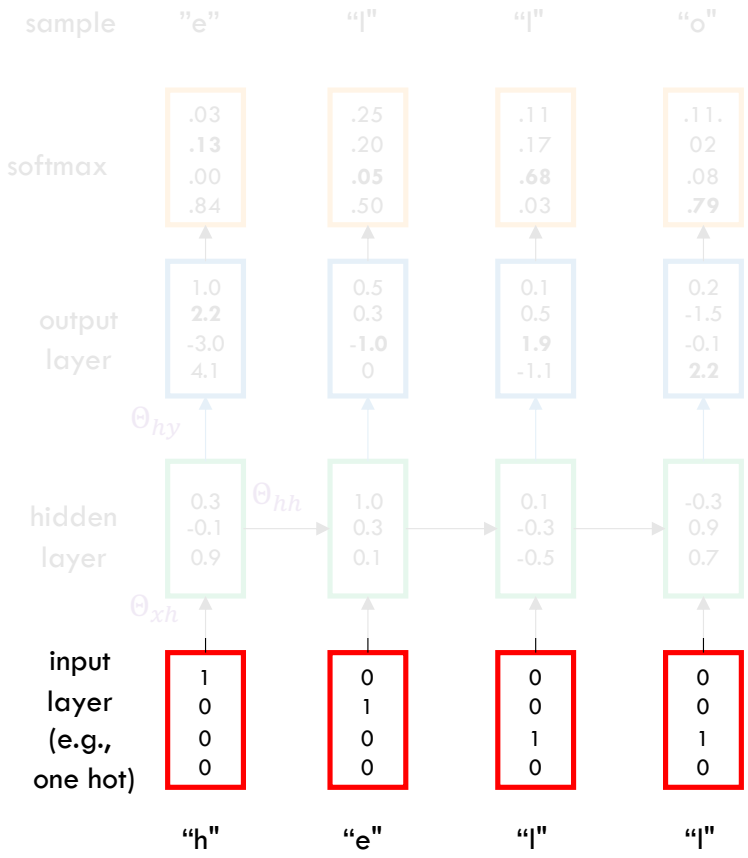
$$h' = g_\Theta(h, x')$$

$$h' = g(\Theta_{hh} h + \Theta_{xh} x')$$

$$y' = \Theta_{hy} h'$$



Example RNN



Character Predictive Language Model

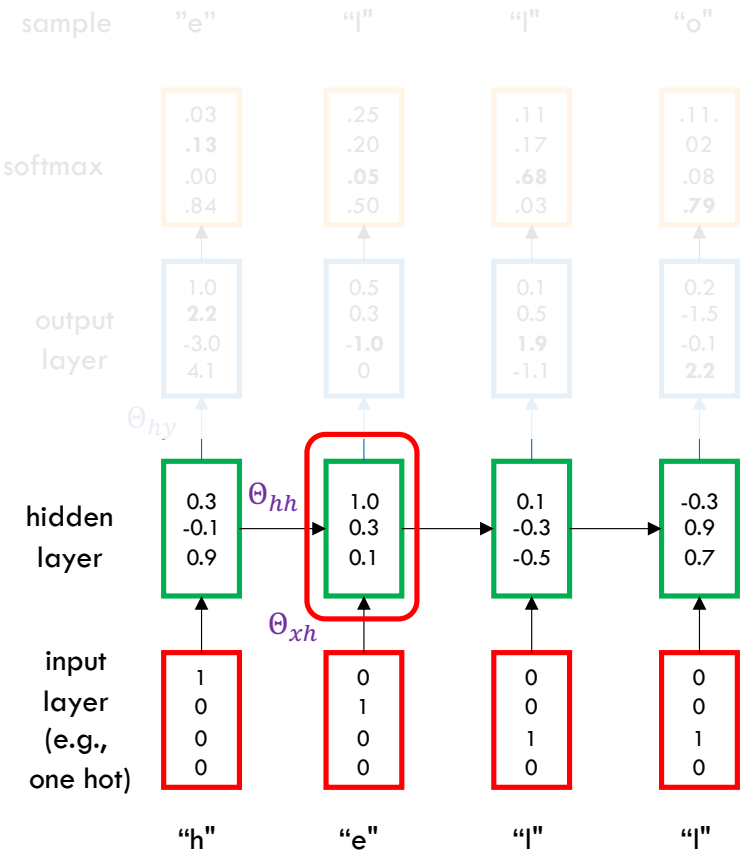
Toy vocabulary [h, e, l, o]

Training: Example training sequence "hello"

Input vector could be vocabulary sized.

Slide credits: Stanford CS231n Lecture 10 Li, Johnson and Yeung

Example RNN



Character Predictive Language Model

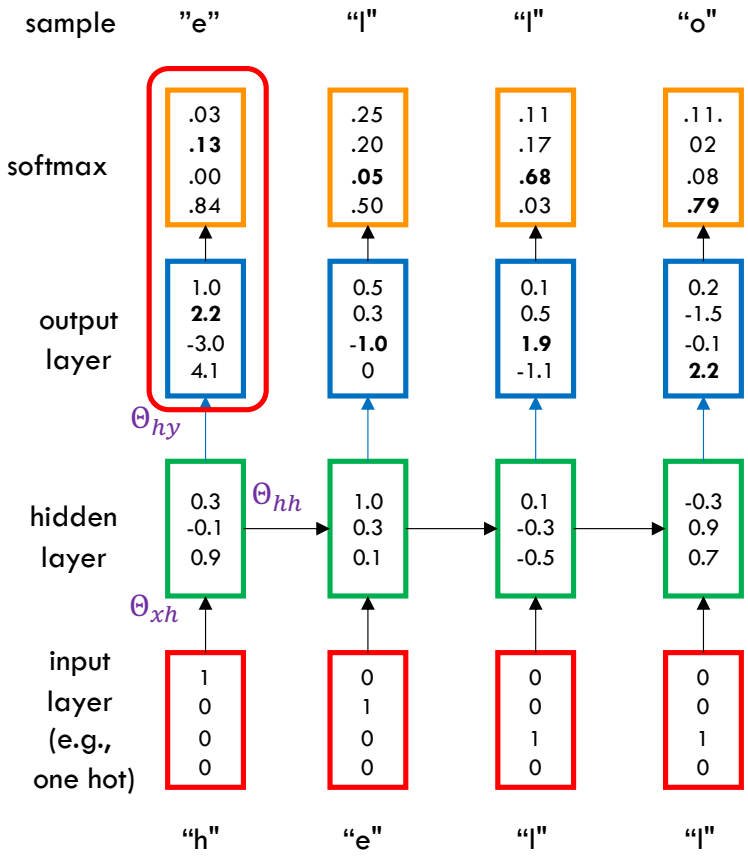
Toy vocabulary [h, e, l, o]

Training: Compute the hidden state going forward:

$$h' = g(\Theta_{hh} h + \Theta_{xh} x')$$

Slide credits: Stanford CS231n Lecture 10 Li, Johnson and Yeung

Example RNN



Character Predictive Language Model

Toy vocabulary [h, e, l, o]

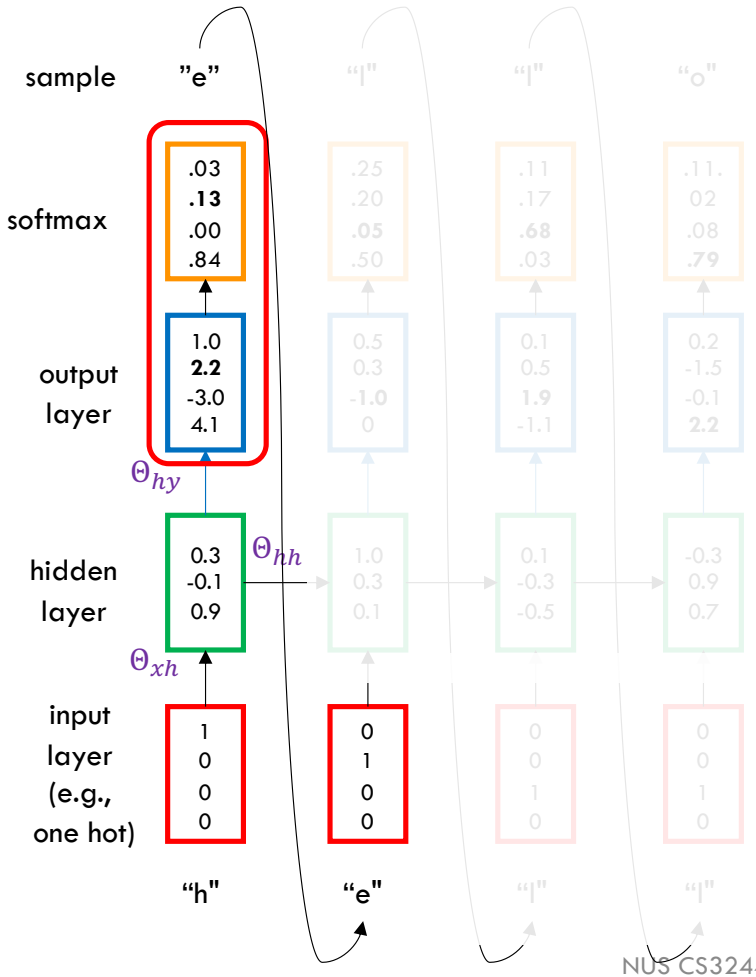
Training: Compute output

$$y = \Theta_{hy} h$$

Propagate loss to train.

Slide credits: Stanford CS231n Lecture 10 Li, Johnson and Yeung

Example RNN



Character Predictive Language Model

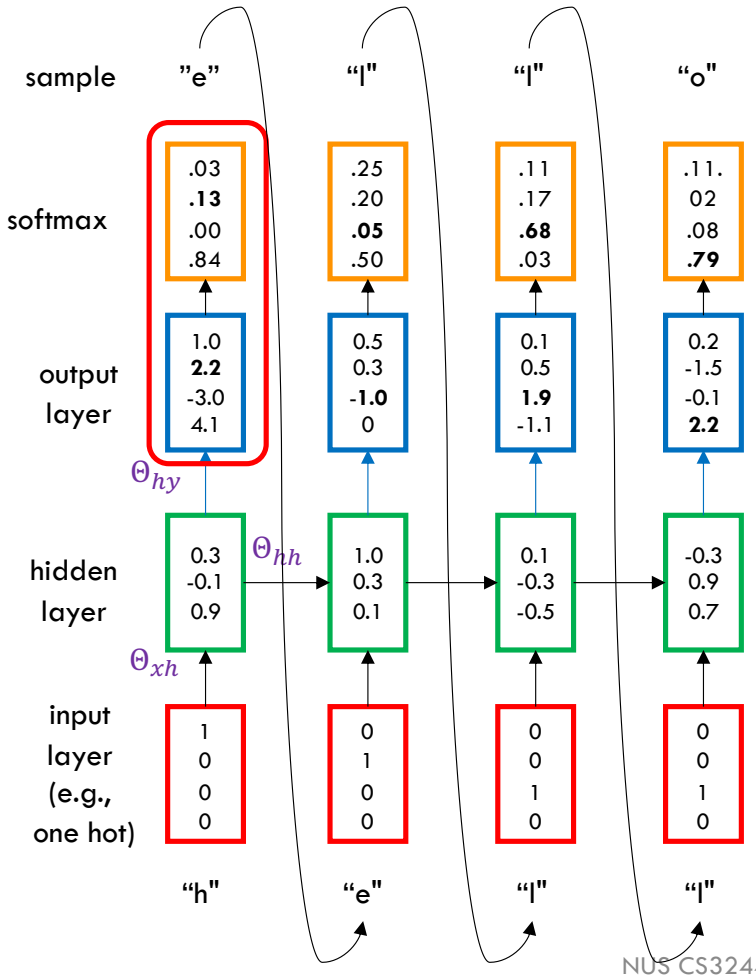
Toy vocabulary [h, e, l, o]

Testing: feed individual characters to the model.

Output of one decision as input to the next.

Slide credits: Stanford CS231n Lecture 10 Li, Johnson and Yeung

Example RNN



Character Predictive Language Model

Toy vocabulary [h, e, l, o]

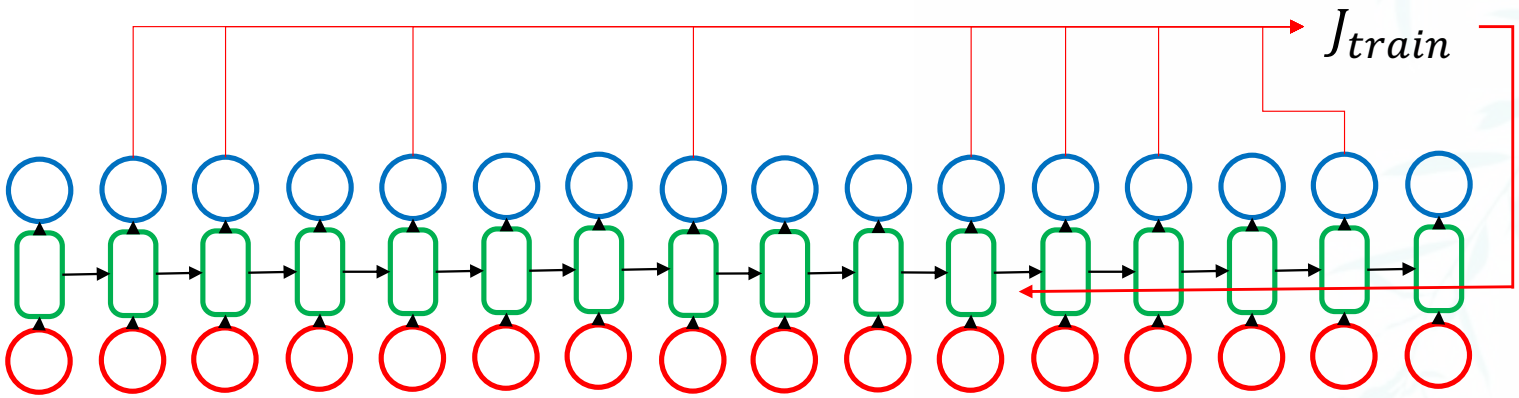
Sample the softmax to get output.

Slide credits: Stanford CS231n Lecture 10 Li, Johnson and Yeung

Backpropagation through time (BTT)

Forward propagate to calculate outputs $h_\Theta(x)$.

Calculate and accumulate loss over all time steps $J_{train} = \sum j(h_\Theta(x), y)$.

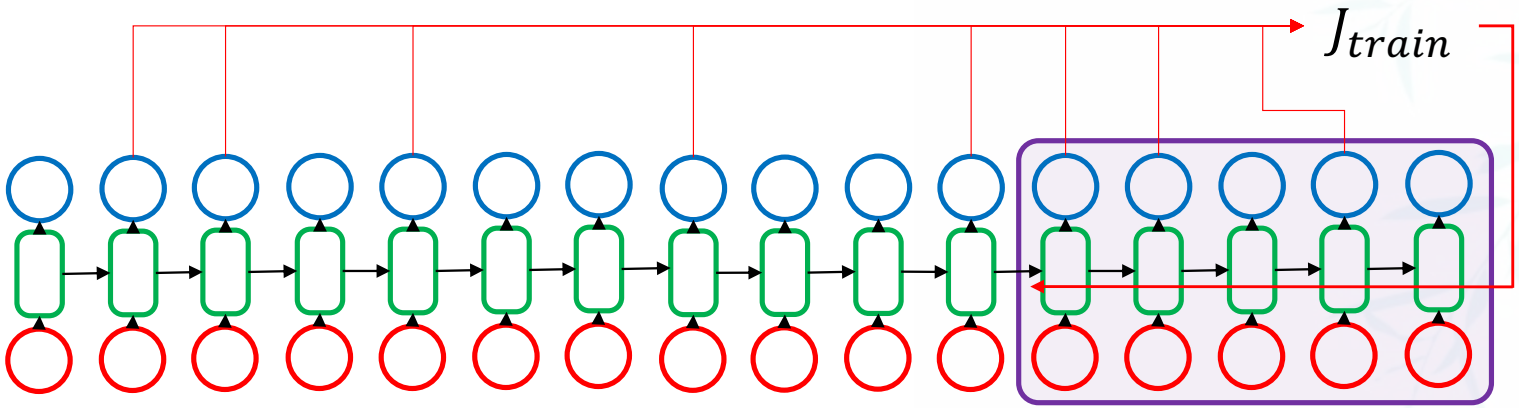


Backpropagate through to the first time step.

Slide credits: Stanford CS231n Lecture 10 Li, Johnson and Yeung

Truncated Backprop through time

Due to gradient problems (mitigated with ReLU) and computational overhead, we may truncate the backprop after k steps.



Some may *partition* k instead of using an actual window k.

Slide credits: Stanford CS231n Lecture 10 Li, Johnson and Yeung